This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

Fast Training of Adversarial Deep Fuzzy Classifier by Downsizing Fuzzy Rules with Gradient Guided Learning

Suhang Gu, *Member, IEEE*, Chi Man Vong, Pak Kin Wong, Shitong Wang

***Abstract*—While our recent deep fuzzy classifier DSA-FC, which stacks adversarial interpretable Takagi-Sugeno-Kang (TSK) fuzzy sub-classifiers, shares its promising classification, its training speed will become very slow and even intolerable for large-scale datasets, due to successive training on all training samples with their random gradient based updates along each layer of its stacked structure. In order to circumvent this bottleneck issue, a fast training algorithm FTA is developed in this study by downsizing fuzzy rules with the proposed gradient guided learning for each sub-classifier at each layer of DSA-FC on large-scale datasets. The core of FTA is to assure fast training of each sub-classifier at each layer of DSA-FC, which first generates first-order smooth gradient guided information by means of the proposed top-*k* fuzzy rules selected from all fuzzy rules in each sub-classifier, and then quickly updates the current inputs in terms of such information, which will be taken as the inputs of the sub-classifier at the next layer. Our theoretical analysis reveals that the proposed gradient guided learning indeed enhances the generalization capability of a deep fuzzy classifier with or without adversarial attacks on outputs. Experimental results on large datasets demonstrate that FTA indeed trains the deep fuzzy classifier DSA-FC quickly with enhanced generalization capability.**

***Index Terms*—Deep fuzzy classifiers, fuzzy rules, adversarial TSK fuzzy classifier, gradient guided learning, large-scale datasets, stacked generalization principle.**

I. Introduction

In our recent work [1], a novel deep fuzzy classifier by stacking adversarial interpretable Takagi-Sugeno-Kang (TSK) fuzzy sub-classifiers with smooth gradient information (DSA-FC) was developed. While recently invented fuzzy and non-fuzzy classification/regression learning techniques have exhibited their promising performance on large-scale data [4,27,31,38], source-target or streaming data [10-12,44-45], as one of deep or wide fuzzy classifiers [9,13,18,34], DSA-FC is trained by the proposed adversarial learning [2-6], where a small percentage of the actual labels of training samples are randomly replaced by the other actual labels [1], the smooth gradient information is then determined at each layer by calculating the first-order gradients on the output error with respect to the inputs of the current layer. Finally, by taking the original inputs plus the smooth gradient information of each layer as the inputs of the next layer, each sub-classifier has its training samples in a layer-by-layer manner. As a result, based on the generalization principle [7-9], DSA-FC has its novel stacked structure by taking the original inputs plus their random gradient based updates of each layer as the inputs of the next layer. Along with the updates of the inputs of each layer by using the smooth gradient information based on the adversarial learning, DSA-FC can lift up its classification performance in a layer-by-layer manner [1]. In addition, owing to the novel stacked structure based on the adversarial learning, not too many fuzzy rules are needed to successively train adversarial TSK fuzzy sub-classifier at each layer of DSA-FC, resulting in better interpretability compared to zero-order or first-order TSK fuzzy classifiers.

However, since the power of DSA-FC in both classification performance and interpretability originates from the successive smooth-gradient-based updates of each training sample by adversarial attacks [1] along with each layer of its stacked structure, its training speed would become of serious bottleneck and even intolerable for training on large-scale datasets. It seems that we can take on-hand dropout-based techniques to speed up the training of DSA-FC. On one hand, according to the concept of *dropout* [19-24,32-33], the classical *dropout* strategy [19] and its recently-developed continuous dropout variants [20-21,24] essentially realize fast training of DSA-FC by activating a certain percentage rather than all of fuzzy rules by some random distribution and then learning their antecedents and consequents in the sub-classifier at current layer of DSA-FC when propagating smooth gradients into training samples of current layer as the inputs of the next layer. On the other hand, due to the random selection of fuzzy rules by dropout-based techniques, smooth gradient information cannot be back-propagated as much as possible such that more comp-

This work was supported in part by National Natural Science Foundation of China under Grants 61772198, 61972181 and U20A20228, by Natural Science Foundation of Jiangsu Province under Grants BK20191331 and 20KJB520023, by National First-class Discipline Program of Light Industry and Engineering (LITE2018), and by the UM (University of Macao) Talent Program. (*Corresponding author*: *Shitong Wang*).

S. Gu is with the School of AI and Computer Science and the Jiangsu Province Key Laboratory of Media Design and Software Technologies, Jiangnan University, Wuxi 214122, China, and also with the School of Electrical Engineering and Automation, Changshu Institute of Technology, Changshu, 215500, China (e-mail: gusuhang09@163.com).

C. M. Vong and P. K. Wong are with the Faculty of Science and Technology, University of Macau, Macau, China (e-mail: cmvong@um.edu.mo, fstpkw@um.edu.mo).

S. Wang is with the School of AI and Computer Science, Jiangnan University, and also with TaiHu Jiangsu Key Construction Laboratory of IoT Application Technology, Wuxi, China (e-mail: wxwangst@aliyun.com).

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

lex stacked structure possibly with more layers are required for satisfactory performance and/or the classification performance is degraded a lot with the same layers. As a result, it becomes naturally motivated for us to invent a novel learning method to share fast training of DSA-FC from the randomness of dropout and simultaneously keep smooth gradient information as much as possible with enhanced generalization capability.

In this study, gradient guided learning is proposed to serve this goal. In order to downsize fuzzy rules to achieve fast training on the sub-classifier at each layer, gradient guided learning only activates top-*k* rather than all fuzzy rules selected by the adopted extreme value distribution instead of some random distribution which is used in the classical dropout, and then calculates the first-order smooth gradients on the output error of the sub-classifier with respect to the training samples. Based on gradient guided learning, a novel fast training algorithm FTA of DSA-FC is developed in this study. Unlike dropout in which randomness in selecting fuzzy rules comes from a random distribution independent of training samples, randomness in selecting top-*k* fuzzy rules by FTA comes from both adversarial attacks and extreme value distribution about the absolute values of the outputs of fuzzy rules, and hence the obtained smooth gradient information becomes more deterministic rather than fully random. We call such randomness *guided randomness* to distinguish this case in this study.

The contributions of this study can be summarized as follows:

(1) Since the proposed fast training algorithm FTA downsizes from all fuzzy rules to top-*k* fuzzy rules in the sub-classifier at each layer by the proposed gradient guided learning, important fuzzy rules are kept as much as possible in training, and accordingly a training simplification of each sub-classifier of DSA-FC is achieved by *guided* rather than *fully* random selection of fuzzy rules. In other words, different from the commonly-used dropout-based techniques [19-24], the selection of top-*k* fuzzy rules indeed provides guided randomness to keep important fuzzy rules in training.

(2) Except for the essence of traditional top-*k* gradient descent method for fast training on neural networks [14-15], as the first attempt, the proposed gradient guided learning theoretically and experimentally demonstrates its very applicability for both more general hidden nodes and outputs with or without adversarial attacks. This is because more general fuzzy rules (i.e., first-order rather than only zero-order fuzzy rules) are adopted in this study while zero-order fuzzy rules can be expressed by general hidden nodes in neural networks [16].

(3) With the help of extreme value distribution about the absolute values of the outputs of fuzzy rules, the proposed gradient guided learning is theoretically justified for its enhanced generalization capability. In other words, we theoretically reveal that the successive selection of top-*k* fuzzy rules by the adopted extreme value distribution can guarantee enhanced generalization capability of the sub-classifier at each layer. This theoretical justification strongly supports the novelty of FTA that it has fast training on DSA-FC and lifts up the interpretability of DSA-FC without the deterioration of the testing performance of DSA-FC.

(4) Comprehensive experiments on benchmark datasets demonstrate that in contrast to the learning algorithm of DSA-FC in [1], FTA has faster training speed and better interpretability under comparable or even better testing accuracies, especially on large-scale datasets.

The reminder of this paper is organized as follows. Section II briefly describes our recent work about DSA-FC. The proposed algorithm FTA and its theoretical justification are given in Section III. Extensive experiments on benchmark datasets are reported in Section IV. The last section concludes this study.

II. Brief Review on DSA-FC

Owing to the use of both stacked structure and adversarial learning, DSA-FC has interpretability guarantee, enhanced classification performance and generalization capability. DSA-FC uses an adversarial first-order TSK fuzzy classifier (TSKa) at each layer of its stacked structure. For the TSKa at each layer, each fuzzy rule takes the following form:

$$\begin{aligned} &Ru^r:\ \text{IF } x_1 \text{ is } A_1^r \wedge x_2 \text{ is } A_2^r \wedge \cdots \wedge x_d \text{ is } A_d^r, \\ &\qquad \text{THEN } f^r(\mathbf{x}) = p_0^r + p_1^r x_1 + \cdots + p_d^r x_d,\ r = 1,2,\ldots,R, \end{aligned} \tag{1}$$

in which R is the total number of fuzzy rules generated in the sub-classifier and Ru^r is the *r*th fuzzy rule. $\mathbf{x} = (x_1, x_2, \cdots, x_d)$ is a *d*-dimensional vector, and A_i^r is the antecedent fuzzy subset corresponding to each x_i in $\mathbf{x}$ for $i = 1,2,\cdots,d$, and it is randomly selected from five fixed fuzzy partitions respectively denoted by appropriate Gaussian fuzzy membership functions with linguistic terms, namely, {*very low, low, medium, high, very high*} [1,9,25]. The symbol " $\wedge$ " denotes a fuzzy conjunction operator. Each fuzzy rule generated in TSKa maps the antecedent fuzzy set $A^r = \{A_1^r, A_2^r, \cdots, A_d^r\}$ into a varying consequent singleton denoted by $f^r(\mathbf{x})$, which is a linear function where $p_0^r, p_1^r, \cdots, p_d^r$ denote the consequent parameters [26-29]. For an input sample $\mathbf{x}$, by using a simple defuzzification strategy [8-9,26-27], the TSKa can produce its output as

$$y^o = \sum_{r=1}^{R} \frac{\mu^r(\mathbf{x})}{\sum_{r'=1}^{R} \mu^{r'}(\mathbf{x})} f^r(\mathbf{x}) = \sum_{r=1}^{R} \tilde{\mu}^r(\mathbf{x}) f^r(\mathbf{x}) \tag{2}$$

where $\mu^r(\mathbf{x})$ is a Gaussian fuzzy membership function corresponding to the *r*th fuzzy rule and its normalized version is expressed as $\tilde{\mu}^r(\mathbf{x})$, which is related to the fuzzy set A^r.

Before training TSKa, a small certain percentage of training samples are randomly selected, and the adversarial attacks will be carried out on their original labels which means the labels are substituted by the other original labels in the original label set denoted by $\bar{\mathbf{Y}}$ and the selected training samples become adversarial samples. After adversarial attacks, $\bar{\mathbf{Y}}$ becomes $\bar{\mathbf{Y}}'$. By training TSKa on the training samples under adversarial attacks, TSKa will assign a label for $\mathbf{x}$ according to Eq. (2).

For TSKa at current layer of DSA-FC, based on the adversarial learning, the smooth gradient information on the output error with respect to the training samples of the current layer can be determined, which will be embedded into the training samp-

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

les at current layer and subsequently the updated training samples will be input into the next TSKa in DSA-FC. In this way, based on stacked generalization principle [1,7-9], the smooth gradient information is embedded into the training samples of each layer in a layer-by-layer manner to consecutively move apart the manifold structure of the original training samples, and each TSKa acquires more generalization capability, resulting in the use of fewer fuzzy rules in each TSKa, compared to the zero-order or first-order TSK fuzzy classifiers. DSA-FC has been theoretically and experimentally indicated its outperformed classification performance while keeping interpretability.

III. The Proposed Algorithm FTA

In this section, we will state the proposed fast training algorithm FTA in detail. In order to achieve fast training of DSA-FC, while keeping the characteristics of DSA-FC such as stacked structure and high interpretability, the basic idea is to determine the smooth gradient information by top-*k* fuzzy rules rather than all fuzzy rules generated in each adversarial TSK fuzzy sub-classifier TSKa before embedding this information into the training samples of each layer in a layer-by-layer manner. By top-*k* fuzzy rules, we mean that only *k* fuzzy rules with the first *k* largest smooth gradients on the output error with respect to the training samples of each sub-classifier TSKa are considered when updating the training samples of each sub-classifier TSKa in DSA-FC. We call such a determination procedure *gradient guided learning*. Since top-*k* fuzzy rules instead of all fuzzy rules are involved in training during each iteration of training, the training of each sub-classifier TSKa at each layer is simplified. As a result, the whole training of DSA-FC becomes so fast, especially on large-scale datasets. Below we will describe the proposed gradient guided learning, justify it theoretically by analyzing the resultant generalization capability, and state the entire implementation of the proposed fast training algorithm FTA.

A. Gradient Guided Learning

Experimental results have demonstrated that the recently-proposed training procedure [1] of DSA-FC becomes so expensive on large-scale datasets, since each fuzzy rule in each sub-classifier TSKa must be updated during each iteration so as to guarantee promising classification performance of DSA-FC. This urges us to study its fast learning mechanism by downsizing fuzzy rules during each iteration of the training of the sub-classifier at each layer of DSA-FC.

A seemingly feasible way is to use the *dropout* technique [19-21] to randomly select some of all fuzzy rules during each iteration of the training for each sub-classifier TSKa. The use of *dropout* can enhance the generalization capability of DSA-FC and speed up the training of DSA-FC. However, since random selection of all the generated fuzzy rules may give up important fuzzy rules, the training accuracy of the resultant deep fuzzy classifier DSA-FC may perhaps be degraded a lot and/or require more layers especially for large-scale datasets. In order to avoid such weakness, a more reliable way should be to take top-*k* fuzzy rules so as to keep important fuzzy rules as much as possible while sharing both enhanced generalization and fast learning speed for DSA-FC, which actually induces gradient guided learning for DSA-FC. Concretely, for each sub-classifier TSKa of DSA-FC, the proposed gradient guided learning calculates the first-order smooth gradients on the output error of the sub-classifier with respect to the training samples, and then takes top-*k* fuzzy rules according to the first *k* largest absolute values of first-order gradients. Since the entire consequent part of each fuzzy rule can be taken as an intermediate variable for the calculation of first-order gradients, the selection of top-*k* fuzzy rules from all the generated fuzzy rules becomes easy.

After each TSKa in DSA-FC is trained in the last iteration, for a training sample $\mathbf{x}=(x_1,x_2,\cdots,x_d)$ in the matrix of $\mathbf{X}=[\mathbf{x}_1,\mathbf{x}_2,\cdots,\mathbf{x}_N]^T$ of training samples at current layer, we can naturally calculate the smooth gradient information $\partial e/\partial\mathbf{x}$ (i.e., the vector $\left(\frac{\partial e}{\partial x_1},\frac{\partial e}{\partial x_2},\cdots,\frac{\partial e}{\partial x_d}\right)$) whose element can be expressed as

$$\frac{\partial e}{\partial x_j}=(y-\bar{y})\frac{\partial y}{\partial x_j} \tag{3}$$

where $j=1,2,\cdots,d$. e denotes the squared loss function $e=(y-\bar{y})^2$ and $\partial y/\partial x_j$ can be further expressed as

$$\begin{aligned}\frac{\partial y}{\partial x_j}=&\sum_{r=1}^{R}\left(\tilde{\mu}^r(\mathbf{x})\sum_{r'=1}^{R}\frac{\tilde{\mu}^{r'}(\mathbf{x})\left(x_j-c_j^{r'}\right)}{\delta_j^{r'}}-\frac{\tilde{\mu}^r(\mathbf{x})\left(x_j-c_j^{r'}\right)}{\delta_j^{r'}}\right)f^r(\mathbf{x})\\&+\sum_{r=1}^{R}\tilde{\mu}^r(\mathbf{x})p_j^r\end{aligned}, \tag{4}$$

in which $f^r(\mathbf{x})$ and $\tilde{\mu}^r(\mathbf{x})$ are defined in the last section, associated with

$$\mu^r(\mathbf{x})=\prod_{j=1}^{d}\mu_{A_j^r}(x_j) \tag{5}$$

where $\mu_{A_j^r}(x_j)$ can be calculated as

$$\mu_{A_j^r}(x_j)=\exp\left(-\left(x_j-c_j^r\right)^2\Big/2\delta_j^r\right), \tag{6}$$

in which c_j^r and δ_j^r are in the parameter vectors $\mathbf{c}^r=(c_1^r,c_2^r,\cdots,c_d^r)$ and $\boldsymbol{\delta}^r=(\delta_1^r,\delta_2^r,\cdots,\delta_d^r)$, and they denote the center of the *r*th cluster, which can be obtained by some clustering strategies such as FCM [27,30], and the width kernel, respectively.

The determination of top-*k* fuzzy rules from all generated fuzzy rules at each layer seems not trivial for us to do it directly from Eq. (3) and Eq. (4). An alternative yet effective way is to determine top-*k* fuzzy rules by using the intermediate variables. According to Eq. (1) and Eq. (2), the squared loss *e* with respect to the input sample $\mathbf{x}$ can be defined as

$$\begin{aligned}e&=\frac{1}{2}\left(\sum_{r=1}^{R}\left(p_0^r+p_1^rx_1+\cdots+p_d^rx_d\right)\tilde{\mu}^r(\mathbf{x})-\bar{y}\right)^2\\&=\frac{1}{2}\left(\sum_{r=1}^{R}\mathbf{p}^r\tilde{\mathbf{x}}^T\tilde{\mu}^r(\mathbf{x})-\bar{y}\right)^2\end{aligned}, \tag{7}$$

in which the vector $\mathbf{p}^r=(p_0^r,p_1^r,\cdots,p_d^r)$ and the vector $\tilde{\mathbf{x}}=[1,\mathbf{x}^T]^T$. According to Eq. (7), our goal actually becomes to select top-*k* fuzzy rules from the total *R* fuzzy rules by observing which *k* fuzzy rules have the first *k* maximum effects on *e*. By

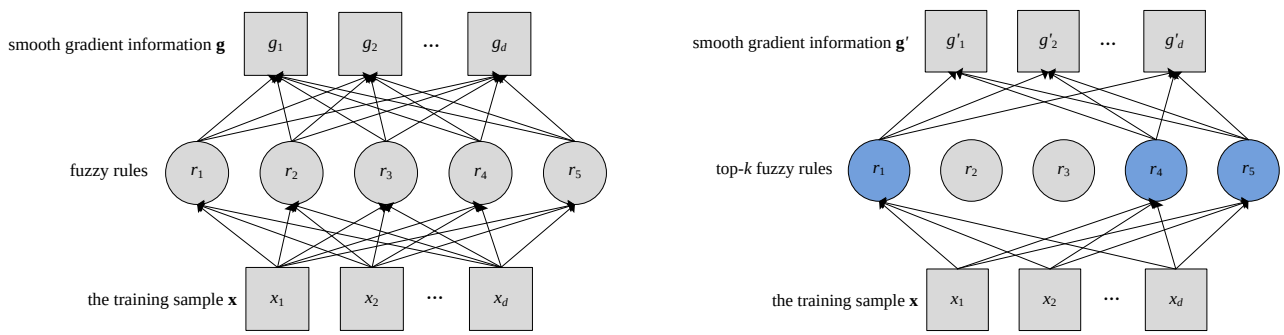
1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

taking $f^r(\mathbf{x})\,(=\mathbf{p}^r\tilde{\mathbf{x}}^T)$ as the *r*th intermediate variable, we can do so using two observational quantities. One quantity is the absolute value of the derivative of *e* with respect to $f^r(\mathbf{x})$, i.e.,

$$\frac{\partial e}{\partial f^r(\mathbf{x})} = \tilde{\mu}^r(\mathbf{x})\left(\sum_{\bar{r}=1}^{R} f^{\bar{r}}(\mathbf{x})\tilde{\mu}^{\bar{r}}(\mathbf{x}) - \bar{y}\right). \tag{8}$$

However, this quantity obviously has a subtle computational burden. The other quantity is the absolute value of $f^r(\mathbf{x})$ itself. According to the construction of DSA-FC in [1], as long as the antecedent parts of the total *R* fuzzy rules are determined at each layer, the absolute value of $f^r(\mathbf{x})$ can be simply taken to observe the contribution of the *r*th fuzzy rule to *e*. We prefer this quantity in this study, owing to both its easy computation and theoretical support in the next subsection. As a result, top-*k* fuzzy rules are determined whose absolute values of $f^r(\mathbf{x})$ fall among the first *k* maximum absolute values of $f^r(\mathbf{x})$ of all fuzzy rules generated in the sub-classifier at each layer.



(a) (b)

Fig. 1. Illustration of the proposed gradient guided learning used at certain layer, in which **x** denotes a training sample, $r_1, r_2, \ldots, r_5$ are the generated five fuzzy rules in the sub-classifier at each layer, **g** is the smooth gradient information vector determined by all the fuzzy rules generated at this layer, and **g**′ is the smooth gradient information vector determined by top-*k* fuzzy rules (denoted as the three blue circles) selected from all the fuzzy rules generated at this layer. (a) Smooth gradient information vector **g** by the learning algorithm in [1]. (b) Smooth gradient information vector **g**′ by the proposed gradient guided learning.

Fig. 1 visually demonstrates the workflow of the proposed gradient guided learning. According to Fig. 1(a) for the training of DSA-FC in [1], we clearly see that when calculating the smooth gradient information vector **g** on the output error with respect to **x** at each layer, all fuzzy rules (denoted as the five grey circles) are taken. However, according to Fig. 1(b) for the proposed gradient guided learning here, only top-*k* fuzzy rules (denoted as the three blue circles) are involved by taking the consequent part, i.e., $f^r(\mathbf{x})$, of each fuzzy rule to calculate the smooth gradient information vector $\mathbf{g}'$ for $\mathbf{x}$. Therefore, the proposed gradient guided learning downsizes from all fuzzy rules to top-*k* fuzzy rules in the sub-classifier at each layer, especially on large-scale datasets which usually needs many fuzzy rules to train the sub-classifier at each layer. In fact, since we may view each hidden node corresponding to each fuzzy rule as a mapping feature, we take $\sqrt{R}$ as the candidate value of *k* in our experiments, referring to the choice of candidate features in [38-39]. Since *k* is generally much less than the total number *R* of fuzzy rules at each layer, it intuitively brings the downsizing effect of fuzzy rules and hence resulting in fast training of the sub-classifier at each layer while keeping high interpretability.

Based on the above analysis, we can further determine the $N \times d$ smooth gradient information matrix **G** where its each element corresponding to training sample $\mathbf{x}_i$ is expressed as

$$g_{ij} = \partial e / \partial x_{ij}\,, \tag{9}$$

in which $i = 1, 2, \cdots, N$, $j = 1, 2, \cdots, d$ and $\partial e/\partial x_{ij}$ is calculated using Eq. (3) and Eq. (4). Please note, $\partial e/\partial x_{ij}$ is only related to top-*k* fuzzy rules rather than all the fuzzy rules generated in the sub-classifier at each layer.

Once the smooth gradient information matrix is determined, the matrix $\mathbf{X}'$ of training samples in the sub-classifier at the next layer are updated as follows

$$\mathbf{X}' = \mathbf{X} + \lambda \mathbf{G}\,, \tag{10}$$

in which the parameter λ denotes the learning rate [1-2], $\mathbf{G} = [\mathbf{g}_1, \mathbf{g}_2, \cdots, \mathbf{g}_N]^T$ where the vector $\mathbf{g}_i = (g_{i1}, g_{i2}, \cdots, g_{id})$, $i = 1, 2, \cdots, N$, and each g_{ij} ($i = 1, 2, \cdots, N$ and $j = 1, 2, \cdots, d$) is given by Eq. (9).

B. Theoretical Justification

When seeking for fast training of DSA-FC by the above gradient guided learning, the testing performance of DSA-FC may generally degrade, due to the use of only top-*k* rather than all fuzzy rules in the sub-classifier at each layer. Therefore, the guarantee of enhanced generalization capability of gradient guided learning should be theoretically justified to offset such performance degradation. In this subsection, following [40-41], we may reasonably suppose that these maximum absolute values follow the extreme value distribution ζ [40-41], and then theoretically analyze the above gradient guided learning in the sense of enhanced generalization capability.

Given the total number *R* of fuzzy rules in the sub-classifier at each layer. When choosing top-*k* fuzzy rules at current layer by calculating the smooth gradient information successively with the extreme value distribution ζ about the absolute value of $f^r(\mathbf{x})$, the corresponding output in Eq. (2) and the squared loss function *e* in Eq. (7) should reflect the successive use of extreme value distribution on the absolute value of $f^r(\mathbf{x})$ in

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

one-by-one way. However, it is not trivial for us to derive a clear analytical expression about the expectation of *e*. What is more, referring to the suggestion that not a little few (quite often a half) hidden nodes (fuzzy rules here) should be kept in dropout-based training, *k* should be bigger than 2 in practice, which is also in accordance with our intuition that if *k*=1 or *k*=2, too much information in all fuzzy rules may be lost, especially for large-scale datasets. Therefore, below our theoretical justification will be based on the analysis about the expectation of *e* in inductive way from *k*=3.

For derivation convenience, we first give the corresponding derivations in the cases of $k=1,2$.

When *k*=1, the term $f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})$ in Eq. (2) becomes $f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\zeta^r$. According to [40-41], the extreme value distribution follows $\zeta \sim (\upsilon,\varsigma)$ where the location parameter $\upsilon=0.577$ and the scale parameter $\varsigma=1.645$ [40-41], thus we can easily have

$$E\left(f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\right)=f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\upsilon, \tag{11}$$

$$\mathrm{Var}\left(f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\right)=\left[f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\right]^2\varsigma. \tag{12}$$

After top-1 (also only one) fuzzy rule works on each sub-classifier at each layer, the output in Eq. (2) with the extreme value distribution ζ can be expressed as

$$y=\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\zeta^r. \tag{13}$$

Therefore, the expectation and variance of the output *y* are respectively expressed as

$$E[y]=\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})E\left[\zeta^r\right]=\upsilon\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x}) \tag{14}$$

and

$$\mathrm{Var}[y]=\sum_{r=1}^{R}\left(f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\right)^2 Var\left[\zeta^r\right]=\varsigma\sum_{r=1}^{R}\left(f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\right)^2. \tag{15}$$

According to Eq. (13), the squared loss function in Eq. (7) can be re-expressed as

$$e=\frac{1}{2}\left(\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\zeta^r-\bar{y}\right)^2. \tag{16}$$

At this moment, the partial derivative of *e* in Eq. (16) with respect to the consequent part of the first fuzzy rule is expressed as

$$\begin{aligned}\frac{\partial e_1}{\partial f^q(\mathbf{x})}&=\tilde{\mu}^q(\mathbf{x})\zeta_1^q\left(\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\zeta_1^r-\bar{y}\right)\\&=\tilde{\mu}^q(\mathbf{x})\zeta_1^q\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\zeta_1^r-\tilde{\mu}^q(\mathbf{x})\zeta_1^q\bar{y}\end{aligned}. \tag{17}$$

Thus we have

$$E\left[\frac{\partial e_1}{\partial f^q(\mathbf{x})}\right]=\upsilon^2\tilde{\mu}^q(\mathbf{x})\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})-\upsilon\tilde{\mu}^q(\mathbf{x})\bar{y}. \tag{18}$$

When $k=2$, it means that after the first fuzzy rule among top-*k* fuzzy rules is determined, the second fuzzy rule with the second maximum absolute value of $f^r(\mathbf{x})$ is taken by following the corresponding extreme value distribution. In this case, the term $f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})$ in Eq. (2) becomes $f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\left(1-\zeta_1^r\right)\zeta_2^r\right)$, and the corresponding squared loss function becomes

$$e_2=\frac{1}{2}\left(\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r-\zeta_1^r\zeta_2^r\right)-\bar{y}\right)^2. \tag{19}$$

The partial derivative of e_2 in Eq. (19) with respect to the consequent part of the second fuzzy rule is accordingly expressed as Eq. (20), as shown at the bottom of the page.

Thus, the expectation of $\partial e_2/\partial f^q(\mathbf{x})$ in Eq. (20) can be readily expressed as

$$\begin{aligned}E\left[\frac{\partial e_2}{\partial f^q(\mathbf{x})}\right]&=\left(2\upsilon-\upsilon^2-\varsigma\right)^2\tilde{\mu}^q(\mathbf{x})\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\\&\quad-\left(2\upsilon-\upsilon^2-\varsigma\right)\tilde{\mu}^q(\mathbf{x})\bar{y}\end{aligned}. \tag{21}$$

When $k=3$, it means that after the first and the second fuzzy rules among top-*k* fuzzy rules are determined, the third fuzzy rule with the third maximum absolute value of $f^r(\mathbf{x})$ is taken by following the corresponding extreme value distribution. In this case, the term $f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})$ in Eq. (2) becomes $f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\left(1-\zeta_1^r\right)\zeta_2^r+\left(1-\zeta_1^r\right)\left(1-\zeta_2^r\right)\zeta_3^r\right)$, and the corresponding squared loss function becomes Eq. (22), as shown at the bottom of the page.

Therefore, the partial derivative of e_3 in Eq. (22) with respect to the consequent part of the third fuzzy rule naturally becomes Eq. (23), as shown at the bottom of the page.

$$\begin{aligned}\frac{\partial e_2}{\partial f^q(\mathbf{x})}&=\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q-\zeta_1^q\zeta_2^q\right)\left(\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r-\zeta_1^r\zeta_2^r\right)-\bar{y}\right)\\&=\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q-\zeta_1^q\zeta_2^q\right)\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r-\zeta_1^r\zeta_2^r\right)-\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q-\zeta_1^q\zeta_2^q\right)\bar{y}\end{aligned}. \tag{20}$$

$$e_3=\frac{1}{2}\left(\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r+\zeta_3^r-\zeta_1^r\zeta_2^r-\zeta_1^r\zeta_3^r-\zeta_2^r\zeta_3^r+\zeta_1^r\zeta_2^r\zeta_3^r\right)-\bar{y}\right)^2. \tag{22}$$

$$\begin{aligned}\frac{\partial e_3}{\partial f^q(\mathbf{x})}&=\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q+\zeta_3^q-\zeta_1^q\zeta_2^q-\zeta_1^q\zeta_3^q-\zeta_2^q\zeta_3^q+\zeta_1^q\zeta_2^q\zeta_3^q\right)\left(\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r+\zeta_3^r-\zeta_1^r\zeta_2^r-\zeta_1^r\zeta_3^r-\zeta_2^r\zeta_3^r+\zeta_1^r\zeta_2^r\zeta_3^r\right)-\bar{y}\right)\\&=\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q+\zeta_3^q-\zeta_1^q\zeta_2^q-\zeta_1^q\zeta_3^q-\zeta_2^q\zeta_3^q+\zeta_1^q\zeta_2^q\zeta_3^q\right)\sum_{r=1}^{R} f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r+\zeta_3^r-\zeta_1^r\zeta_2^r-\zeta_1^r\zeta_3^r-\zeta_2^r\zeta_3^r+\zeta_1^r\zeta_2^r\zeta_3^r\right)\\&\quad-\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q+\zeta_3^q-\zeta_1^q\zeta_2^q-\zeta_1^q\zeta_3^q-\zeta_2^q\zeta_3^q+\zeta_1^q\zeta_2^q\zeta_3^q\right)\bar{y}\end{aligned}. \tag{23}$$

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

Thus, the expectation of $\partial e_3/\partial f^q(\mathbf{x})$ in Eq. (23) becomes

$$E\left[\frac{\partial e_3}{\partial f^q(\mathbf{x})}\right]=\left(\upsilon^3-3\upsilon^2+3\upsilon+3\upsilon\varsigma-3\varsigma\right)^2\tilde{\mu}^q(\mathbf{x})\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})-\left(\upsilon^3-3\upsilon^2+3\upsilon+3\upsilon\varsigma-3\varsigma\right)\tilde{\mu}^q(\mathbf{x})\bar{y}. \quad (24)$$

When $k=4$, the term $f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})$ in Eq. (2) becomes $f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\left(1-\zeta_1^r\right)\zeta_2^r+\left(1-\zeta_1^r\right)\left(1-\zeta_2^r\right)\zeta_3^r+\left(1-\zeta_1^r\right)\left(1-\zeta_2^r\right)\left(1-\zeta_3^r\right)\zeta_4^r\right)$, and the corresponding squared loss function becomes Eq. (25), as shown at the bottom of the page.

In this case, the partial derivative of e_4 in Eq. (25) with respect to the consequent part of the fourth fuzzy rule becomes Eq. (26), as shown at the bottom of the page.

Thus, the expectation of $\partial e_4/\partial f^q(\mathbf{x})$ in Eq. (26) becomes

$$E\left[\frac{\partial e_4}{\partial f^q(\mathbf{x})}\right]=\left(-\upsilon^4+4\upsilon^3-6\upsilon^2-6\upsilon^2\varsigma+4\upsilon+12\upsilon\varsigma-3\varsigma^2-6\varsigma\right)^2\tilde{\mu}^q(\mathbf{x})\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})-\left(-\upsilon^4+4\upsilon^3-6\upsilon^2-6\upsilon^2\varsigma+4\upsilon+12\upsilon\varsigma-3\varsigma^2-6\varsigma\right)\tilde{\mu}^q(\mathbf{x})\bar{y}. \quad (27)$$

Due to the paper's space limitation, we omit the corresponding subtle derivations about bigger k values. Even so, according to Eqs. (18), (21), (24) and (27), we can generalize the expectation of $\partial e_k/\partial f^q(\mathbf{x})$ about top-k fuzzy rules as follows

$$E\left[\frac{\partial e}{\partial f^q(\mathbf{x})}\right]=a\tilde{\mu}^q(\mathbf{x})\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})+b\tilde{\mu}^q(\mathbf{x})\bar{y}. \quad (28)$$

Since $\upsilon=0.577$ and $\varsigma=1.645$ [40-41], after simple calculations, the coefficients a of $\tilde{\mu}^q(\mathbf{x})\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})$ in Eqs. (18), (21), (24) and (27) are about 0.33, 0.68, 1.35 and 79.50, respectively, which means that a becomes bigger and bigger (than 1) from $k>2$.

As for DSA-FC, the smooth gradient information are involved in all fuzzy rules generated in TSKa at each layer. Thus, the squared loss function e' of the corresponding sub-classifier TSKa can be expressed as

$$e'=\frac{1}{2}\left(\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})-\bar{y}\right)^2. \quad (29)$$

Similarly, the partial derivative of e' with respect to $f^q(\mathbf{x})$ is expressed as

$$\frac{\partial e'}{\partial f^q(\mathbf{x})}=\tilde{\mu}^q(\mathbf{x})\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})-\tilde{\mu}^q(\mathbf{x})\bar{y}. \quad (30)$$

The expectation of $\partial e'/\partial f^q(\mathbf{x})$ in Eq. (30) accordingly be

$$E\left[\frac{\partial e'}{\partial f^q(\mathbf{x})}\right]=\tilde{\mu}^q(\mathbf{x})\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})-\tilde{\mu}^q(\mathbf{x})\bar{y}. \quad (31)$$

Comparing Eq. (28) to Eq. (31), we can derive the following relationship between $\partial e/\partial f^q(\mathbf{x})$ and $\partial e'/\partial f^q(\mathbf{x})$:

$$E\left[\frac{\partial e}{\partial f^q(\mathbf{x})}\right]=E\left[\frac{\partial e'}{\partial f^q(\mathbf{x})}\right]+(a-1)f^q(\mathbf{x})\left(\tilde{\mu}^q(\mathbf{x})\right)^2+(a-1)\tilde{\mu}^q(\mathbf{x})\sum_{r=1,r\neq q}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})+(b+1)\tilde{\mu}^q(\mathbf{x})\bar{y}. \quad (32)$$

Therefore, we accordingly have

$$\bar{E}[e]=e'+\frac{(a-1)}{2}\left(f^q(\mathbf{x})\tilde{\mu}^q(\mathbf{x})\right)^2+(a-1)f^q(\mathbf{x})\tilde{\mu}^q(\mathbf{x})\bullet\sum_{r=1,r\neq q}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})+(b+1)f^q(\mathbf{x})\tilde{\mu}^q(\mathbf{x})\bar{y}+\text{const}. \quad (33)$$

By accumulating both sides of Eq. (33) with respect to $q=1,2,\cdots,R$, the relationship between e and e' can be further expressed as

$$E[e]=e'+\frac{(a-1)}{2R}\sum_{r=1}^{R}\left(f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\right)^2+\frac{(a-1)}{R}\sum_{r'=1,r'\neq r}^{R}\sum_{r=1}^{R}f^{r'}(\mathbf{x})\tilde{\mu}^{r'}(\mathbf{x})f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})+(b+1)\bar{y}\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})+\text{const}. \quad (34)$$

According to Eq. (34), top-k fuzzy rules in the sub-classifier at each layer actually cause three additional regularization terms. Specially, when $k>2$, as seen in the above, $a>1$. Therefore, whatever $\bar{y}$ has an adversarial attack or not and b is positive or negative, the second and the fourth terms in Eq. (34) indeed form a convex function about $f^q(\mathbf{x})$, which actually aim to provide extra generalization capability for $f^q(\mathbf{x})$ ($q=1$,

$$e_4=\frac{1}{2}\left(\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r+\zeta_3^r+\zeta_4^r-\zeta_1^r\zeta_2^r-\zeta_1^r\zeta_3^r-\zeta_1^r\zeta_4^r-\zeta_2^r\zeta_3^r-\zeta_2^r\zeta_4^r-\zeta_3^r\zeta_4^r+\zeta_1^r\zeta_2^r\zeta_3^r+\zeta_1^r\zeta_2^r\zeta_4^r+\zeta_1^r\zeta_3^r\zeta_4^r+\zeta_2^r\zeta_3^r\zeta_4^r-\zeta_1^r\zeta_2^r\zeta_3^r\zeta_4^r\right)-\bar{y}\right)^2. \quad (25)$$

$$\begin{aligned}\frac{\partial e_4}{\partial f^q(\mathbf{x})}&=\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q+\zeta_3^q+\zeta_4^q-\zeta_1^q\zeta_2^q-\zeta_1^q\zeta_3^q-\zeta_1^q\zeta_4^q-\zeta_2^q\zeta_3^q-\zeta_2^q\zeta_4^q-\zeta_3^q\zeta_4^q+\zeta_1^q\zeta_2^q\zeta_3^q+\zeta_1^q\zeta_2^q\zeta_4^q+\zeta_1^q\zeta_3^q\zeta_4^q+\zeta_2^q\zeta_3^q\zeta_4^q-\zeta_1^q\zeta_2^q\zeta_3^q\zeta_4^q\right)\\&\bullet\left(\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r+\zeta_3^r+\zeta_4^r-\zeta_1^r\zeta_2^r-\zeta_1^r\zeta_3^r-\zeta_1^r\zeta_4^r-\zeta_2^r\zeta_3^r-\zeta_2^r\zeta_4^r-\zeta_3^r\zeta_4^r+\zeta_1^r\zeta_2^r\zeta_3^r+\zeta_1^r\zeta_2^r\zeta_4^r+\zeta_1^r\zeta_3^r\zeta_4^r+\zeta_2^r\zeta_3^r\zeta_4^r-\zeta_1^r\zeta_2^r\zeta_3^r\zeta_4^r\right)-\bar{y}\right)\\&=\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q+\zeta_3^q+\zeta_4^q-\zeta_1^q\zeta_2^q-\zeta_1^q\zeta_3^q-\zeta_1^q\zeta_4^q-\zeta_2^q\zeta_3^q-\zeta_2^q\zeta_4^q-\zeta_3^q\zeta_4^q+\zeta_1^q\zeta_2^q\zeta_3^q+\zeta_1^q\zeta_2^q\zeta_4^q+\zeta_1^q\zeta_3^q\zeta_4^q+\zeta_2^q\zeta_3^q\zeta_4^q-\zeta_1^q\zeta_2^q\zeta_3^q\zeta_4^q\right)\\&\bullet\sum_{r=1}^{R}f^r(\mathbf{x})\tilde{\mu}^r(\mathbf{x})\left(\zeta_1^r+\zeta_2^r+\zeta_3^r+\zeta_4^r-\zeta_1^r\zeta_2^r-\zeta_1^r\zeta_3^r-\zeta_1^r\zeta_4^r-\zeta_2^r\zeta_3^r-\zeta_2^r\zeta_4^r-\zeta_3^r\zeta_4^r+\zeta_1^r\zeta_2^r\zeta_3^r+\zeta_1^r\zeta_2^r\zeta_4^r+\zeta_1^r\zeta_3^r\zeta_4^r+\zeta_2^r\zeta_3^r\zeta_4^r-\zeta_1^r\zeta_2^r\zeta_3^r\zeta_4^r\right)\\&-\tilde{\mu}^q(\mathbf{x})\left(\zeta_1^q+\zeta_2^q+\zeta_3^q+\zeta_4^q-\zeta_1^q\zeta_2^q-\zeta_1^q\zeta_3^q-\zeta_1^q\zeta_4^q-\zeta_2^q\zeta_3^q-\zeta_2^q\zeta_4^q-\zeta_3^q\zeta_4^q+\zeta_1^q\zeta_2^q\zeta_3^q+\zeta_1^q\zeta_2^q\zeta_4^q+\zeta_1^q\zeta_3^q\zeta_4^q+\zeta_2^q\zeta_3^q\zeta_4^q-\zeta_1^q\zeta_2^q\zeta_3^q\zeta_4^q\right)\bar{y}\end{aligned}. \quad (26)$$

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

$2,\cdots,R$). In other words, this observation actually reveals that while DSA-FC has its enhanced generalization capability by means of adversarial attacks on outputs [1], the gradient guided learning here may provide at least comparable or even more generalization capability. What is more, the third term in Eq. (34) actually aims to minimize the co-adaption capability between each fuzzy rule generated in the sub-classifier at each layer. In other words, with the above derivations, we can have the theoretical justification that when $k>2$, the use of top-*k* fuzzy rules not only provides extra generalization capability, but also brings extra benefit in avoiding the co-adaption between each fuzzy rule.

What is more, according to the above mathematical derivations, when $\tilde{\mu}^r(\mathbf{x})$ is replaced with the commonly used sigmoid function and $f^r(\mathbf{x})$ is taken as zero-order consequent part (i.e., $f^r(\mathbf{x})=\mathbf{p}^r\tilde{\mathbf{x}}^T=p_0^r$ (i.e., $p_1^r=p_2^r=\cdots=p_d^r=0$)) or high-order instead of first-order consequent part [18], we can easily see that the above analysis still holds. This actually means that top-*k* gradient descent is also justified as the first theoretical attempt for deep neural networks, and the above claims hold for deep high-order fuzzy classifiers.

C. Fast Training Procedure

In this subsection, we will first state the entire fast training procedure of FTA and then describe how to do prediction for a testing sample by using the resultant deep fuzzy classifier on this testing sample.

While the above gradient guided learning is taken to update all original training samples as the inputs of the sub-classifier at the next layer, due to the use of only top-*k* fuzzy rules under adversarial attacks, a training simplification of the sub-classifier at each layer actually appears. In order to keep good trade-off between such a training simplification and the guarantee of training accuracy, after considering randomness from adversarial attacks, we may do such a training simplification under adversarial attacks for several times (e.g., three times in our experiments) at each layer, and then take the corresponding average training results. Even so, our experimental results will reveal that the whole training of DSA-FC based on the above gradient guided learning is still speeded up. Except this, the proposed fast training algorithm FTA keeps the same training steps as in DSA-FC. In addition, each feature of each training sample is mapped into the five fuzzy partitions with linguistically interpretable terms, e.g., {*very low*, *low*, *medium*, *high*, *very high*}. Owing to the novel stacked structure of DSA-FC, not too many fuzzy rules are involved by FTA on the sub-classifier at each layer. As a result, FTA still can guarantee promising performance and high interpretability of DSA-FC.

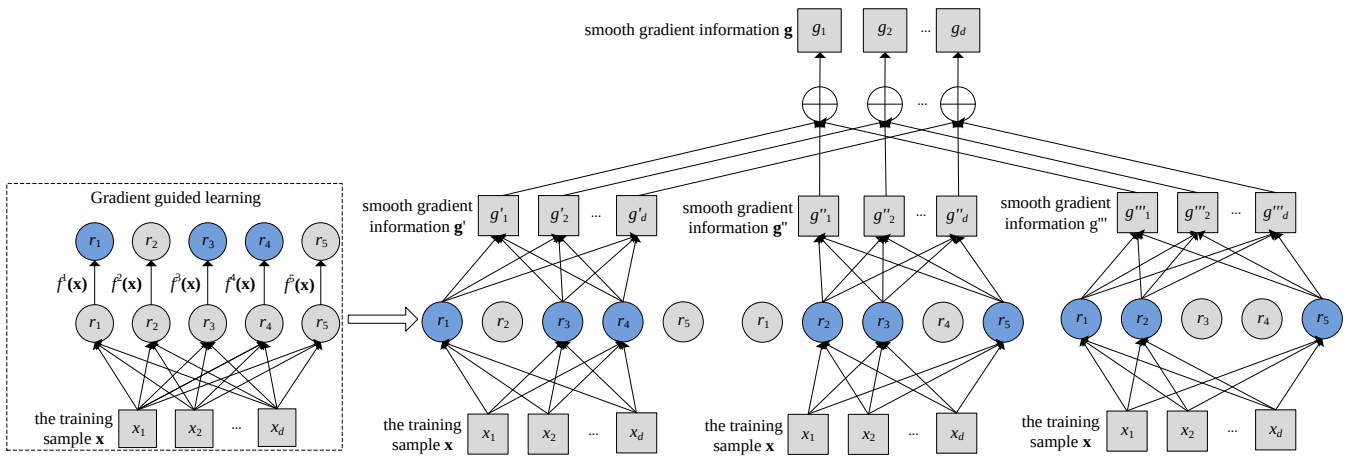


Fig. 2. An illustrative training workflow of FTA on the sub-classifier for three times by FTA at one certain layer, where all the generated fuzzy rules are denoted by $r_1,r_2,\cdots,r_5$, top-*k* fuzzy rules are denoted by blue circles. $\mathbf{x}$ denotes a training sample which will be updated by the smooth gradient information based on the proposed gradient guided learning. The smooth gradient information vector $\mathbf{g}$ expresses the average results of smooth gradient information vectors $\mathbf{g}'$, $\mathbf{g}''$ and $\mathbf{g}'''$ obtained by training on the sub-classifier with top-*k* fuzzy rules for three times.

Fig. 2 visually presents the training workflow of FTA on the sub-classifier at certain layer. According to Fig. 2, FTA first selects top-*k* fuzzy rules (denoted by the three blue circles) from all generated fuzzy rules (denoted by the five grey circles) using the above gradient guided learning. Then it trains the sub-classifier at this layer in the same way for three times, which actually means that three sub-classifiers are obtained. We can clearly see from Fig. 2 that after smooth gradient information vectors (respectively denoted as $\mathbf{g}'$, $\mathbf{g}''$ and $\mathbf{g}''$) are calculated, according to Eq. (9), FTA uses their average results (denoted by $\mathbf{g}$) to update the training samples at this layer as the inputs of the next layer. This significantly distinguishes FTA from the learning algorithm of DSA-FC in [1] where all the generated fuzzy rules at this layer are used for the training of the sub-classifier at this layer.

The whole training procedure of FTA is very similar to that of DSA-FC in [1]. Fig. 3 illustrates the entire training workflow of the proposed fast training algorithm FTA. According to Fig. 3, by using gradient guided leaning at each layer, FTA begins with the training of the first sub-classifier at the first layer on the original training samples. After that, FTA generates each input of the second sub-classifier as each original training sample

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

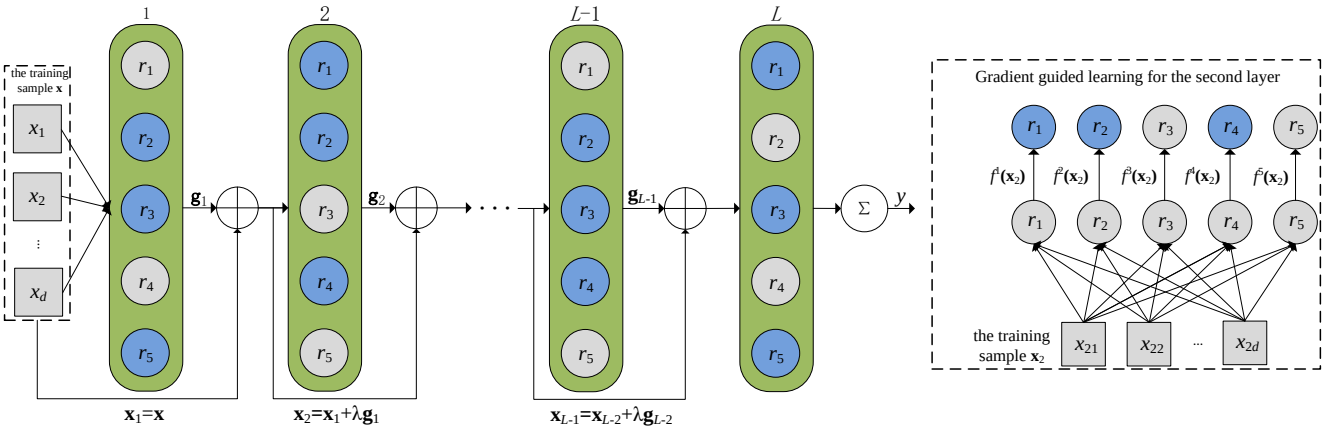


Fig. 3. The training workflow of FTA on the deep fuzzy classifier with L layers.

plus smooth gradient information vector $\mathbf{g}_1$ determined by top-k fuzzy rules based on the proposed gradient guided learning from the first layer, as shown in the dot-line subfigure in Fig. 3. In the same way, FTA works for the second sub-classifier and generates each training sample plus its corresponding smooth gradient information vector $\mathbf{g}_2$ at the second layer as the inputs of the third sub-classifier. Repeat the above procedure until satisfactory performance is achieved or the maximum number of layers is met.

The whole training procedure of FTA is summarized in Algorithm 1. Please note, we simply call the original learning algorithm 1 in [1] to train each sub-classifier at each layer.

After the resultant deep fuzzy classifier is obtained by Algorithm 1, we can readily use it to make prediction for a testing sample by presenting it as the input into each sub-classifier in the resultant deep fuzzy classifier and then deciding its label by taking majority-voting or averaging strategy on the corresponding outputs of all the sub-classifiers. We take the averaging strategy in our experiments.

Algorithm 1: The entire fast training procedure.

Input: The matrix of original training dataset $\mathbf{X}=\left[\mathbf{x}_1,\mathbf{x}_2,\cdots,\mathbf{x}_N\right]^T$, where $\mathbf{x}_i \in R^d$ in which d represents the dimensions; the actual label vector $\bar{\mathbf{Y}}=\left[\bar{y}_1,\bar{y}_2,\cdots,\bar{y}_N\right]^T \in R^N$; the maximum number L of layers; the learning rate λ; the chosen percentage S for adversarial attacks on $\bar{\mathbf{Y}}$ [1]; the given number of fuzzy rules being $R_1, R_2, \cdots, R_L$, respectively, for each sub-classifier at each layer; the number M of the training for each sub-classifier.

Output: The architecture of the resultant deep fuzzy classifier and each sub-classifier therein.

Procedure:

Step 1: Let $\mathbf{X}_1=\mathbf{X}$ and $l=1$.

Step 2: Present $\mathbf{X}_l$ and $\bar{\mathbf{Y}}$ as the inputs to the sub-classifier at lth layer in [1];

Step 3: $m=1$

Repeat

Call Algorithm 1 in [1] to train the lth sub-classifier TSKa;

Calculate the absolute value of $f^r(\mathbf{x})$ for the rth fuzzy rule with the ith input sample $\mathbf{x}_i$, where $r=1,2,\cdots,R_l$ and $i=1,2,\cdots,N$;

Determine the top-k fuzzy rules by fixing the first k maximum absolute values of $f^r(\mathbf{x})$ from the total R_l fuzzy rules with all training samples, where $k=\sqrt{R_l}$;

Determine the $N\times d$ smooth gradient information matrix $\mathbf{G}_{l,m}$ where each element is calculated using Eq. (9);

$m=m+1$

Until $m>M$, and then output the smooth gradient information matrix $\mathbf{G}_l=\left(\mathbf{G}_{l,1}+\mathbf{G}_{l,2}+\cdots+\mathbf{G}_{l,M}\right)/M$.

Step 4: If satisfactory performance is achieved in terms of training accuracy or the maximum number L of layers has been met, then terminate; otherwise $\mathbf{X}_{l+1}=\mathbf{X}_l+\lambda\mathbf{G}_l$.

Step 5: $l \leftarrow l+1$, return to Step 2.

IV. Experimental Results and Analysis

In this section, we will give an insight into the performance of FTA. Because we design FTA to speed up the training of our recently-proposed deep fuzzy classifier DSA-FC [1], the original learning algorithm of DSA-FC is naturally taken as the compared algorithm. Since we did not abbreviate the original learning algorithm of DSA-FC, here we denote it as DSA-FC for simplicity, with a slight abuse of the abbreviation word DSA-FC. In addition, since the proposed gradient guided learning behaves like dropout learning [19], the deep neural networks with Bernoulli dropout [19] as well as the

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

state-of-the-art continuous dropout, i.e., Gaussian dropout [21], are taken as the comparison algorithms. Please note, we denote the adopted neural networks with Bernoulli dropout and Gaussian dropout as *Bernoulli dropout* and *Gaussian dropout*, respectively, hereafter for simplicity. In order to observe the performance differences between FTA and all the adopted algorithms (especially DSA-FC), eighteen real-world datasets are adopted. Just likewise in [1], in order to examine generalization capabilities and training speeds of the learning algorithms, our observation will mainly be focused on the testing accuracies and training time of the learning algorithms. In order to make our comparative study fair, we conduct each learning algorithm on each adopted dataset for ten runs, and then report the corresponding average results. Also, during each run, as for FTA, the sub-classifier at each layer of the corresponding deep fuzzy classifier is built three times to record the corresponding average results.

A. Experimental Setup

TABLE I
DESCRIPTIONS OF THE ADOPTED DATASETS

#	Datasets	#Sizes	#Features	#Classes
1	*Waveform(WAV)*	5000	21	3
2	*Phoneme(PHO)*	5404	5	2
3	*Musk(MUS)*	6598	166	2
4	*ElectricalGridStability(EGS)*	10000	13	2
5	*Nursery(NUR)*	12960	8	5
6	*HTRU2(HTR)*	17898	8	2
7	*Avila(AVI)*	20867	10	12
8	*CreditCardClients(CCC)*	30000	23	2
9	*Adult(ADU)*	48841	14	2
10	*Shuttle(SHU)*	58000	9	8
11	*InternetFirewall(INF)*	65532	11	4
12	*Connect(CON)*	67757	42	3
13	*ConfLongDemo(CLD)*	114434	3	3
14	*ClickStream(CLS)*	165474	13	4
15	*SkinSegmentation(SKI)*	245057	3	2
16	*Covertype(COV)*	495126	54	2
17	*Kddcup(KDD)*	494020	41	23
18	*Poker(POK)*	946798	10	2

All the adopted real-world datasets can be downloaded from the UCI [36] and KEEL [37] data repositories, and their details are summarized in Table I. According to Table I, the number of data samples ranges from 5000 to 946798, the number of features from 3 to 166, and the number of data classes from 2 to 23. Since the number of the data samples in each of the first eight datasets are less than 30000, as for DSA-FC and FTA, the number R of fuzzy rules generated in the sub-classifier at each layer is simply optimized over the set $\{2,3,\cdots,9,10\}$. As for the remaining ten large-scale datasets, the number R of fuzzy rules generated in the sub-classifier at the first layer is initially set to be a comparatively big value. According to the powerful stacked generalization principle [1,7-9], the number R of fuzzy rules generated in the remaining sub-classifiers may gradually decrease along with the corresponding structures. More details about the parameter R for these ten datasets are summarized in Table II. Take the dataset *SKI* as an example, the sub-classifier at the first layer optimizes the number R of fuzzy rules over $\{50,60,\cdots,140,150\}$. Along with the sub-classifiers at the subsequent layers, R's search ranges decrease gradually with the interval being 10. That is to say, with this interval, the search range at the second layer becomes $\{40,50,\cdots,130,140\}$, and for the third layer, the search range becomes $\{30,40,\cdots,120,130\}$. This procedure continues until the best performance or the maximum number of layers is achieved. As for Bernoulli dropout [19] and Gaussian dropout [21], for fair comparison, the number of hidden layers is comparably set to be the number of layers of the deep fuzzy classifier by FTA, and the number of hidden nodes in each hidden layer is also comparably set to be the number of fuzzy rules used in the sub-classifier at each layer of the deep fuzzy classifier by FTA.

TABLE II
NUMBER OF FUZZY RULES IN THE SUB-CLASSIFIER AT EACH LAYER ON THE TEN DATASETS

Datasets	Ranges	Intervals
ADU	50:5:100	5
SHU	50:5:100	10
INF	50:5:100	10
CON	100:10:220	10
CLD	50:10:150	10
CLS	50:10:150	10
SKI	50:10:150	10
COV	30:5:150	5
KDD	30:5:150	5
POK	50:10:250	10

As for both DSA-FC and FTA, the involved parameters are the number R of fuzzy rules and the regularization parameter τ in the first-order fuzzy sub-classifier at each layer, the percentage S for adversarial attacks on actual label sets of the given datasets and the learning rate λ [1-2] in Eq. (10) for updating the inputs of each layer. The regularization parameter τ is optimized over the set $\{10^{-5},10^{-4},\cdots,10^{4},10^{5}\}$ according to [27,35], and the parameters S and λ adopt the recommended values, i.e., S=0.05(5%) and $\lambda = 0.001$ according to [1]. All the hyperparameters of Bernoulli and Gaussian dropouts are set to follow [17,21]. Besides, all the parameters are fairly optimized for each adopted algorithm by the grid search strategy with a stratified ten-fold cross-validation. For each adopted dataset, eighty percentage of all the samples are randomly taken as training dataset, and the remaining ones as testing dataset. All experiments in this study are conducted on Matlab R2015b on a desktop computer equips with Intel(R) Core(TM) i7-8700K CPU with 3.20GHz frequency, 64G memory space, and 64 bit Windows 10 Ultimate System.

B. Performance on Real-world Datasets

This subsection presents comparative study of the proposed learning algorithm FTA against our recent deep learning algorithm DSA-FC [1], Bernoulli dropout and Gaussian dropout [19,21] on eighteen real-world datasets. Table III shows the average testing accuracies of the adopted algorithms on the adopted real-world datasets. In Table III, the average testing accuracy is denoted as *Acc*±*Std* where *Acc* denotes the average testing accuracy and *Std* denotes the corresponding standard deviation. "--" denotes the standard deviation being

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

smaller than 10^{-4}. $\bar{R}$ and L denote the total number of fuzzy rules and the total number of the layers in the deep fuzzy classifiers built by both DSA-FC and FTA, respectively. R_l denotes the total number of fuzzy rules in the sub-classifier at each layer of the deep fuzzy classifier by DSA-FC, and k denotes the total number of top-k fuzzy rules at each layer of the deep fuzzy classifier by FTA.

The testing accuracies listed in Table III indeed verify that Gaussian dropout can outperform Bernoulli dropout in most cases [21]. Besides, FTA outperforms Gaussian dropout on 11 out of 18 real-world datasets. This is because top-k fuzzy rules are selected by the proposed gradient guided learning while both Bernoulli and Gaussian dropouts essentially choose hidden nodes randomly. After FTA is compared to both Bernoulli and Gaussian dropouts, below our experimental organization mainly aims at observing/examining whether FTA generates less fuzzy rules (hence better generalization capability and interpretability) than DSA-FC in the resultant deep fuzzy classifiers under comparable or even better testing performances on all the adopted datasets.

TABLE III
AVERAGE TESTING ACCURACIES ON REAL-WORLD DATASETS BY BOTH ADOPTED ALGORITHMS

Datasets	DSA-FC				FTA				Bernoulli dropout	Gaussian dropout
	Acc	$(\bar{R},\tau)$	*L*	R_l	*Acc*	$(\bar{R},\tau)$	*L*	*k*	*Acc*	*Acc*
WAV	68.73±0.90	$(37.8,10^{-3})$	4	9.8-9.8-9.2-9.0	66.79±2.11	$(18,10^{-4})$	2	3-3	71.80±--	71.90±--
PHO	79.40±0.54	$(48.2,10^{-5})$	5	9.8-9.8-9.8-10.0-8.8	76.48±0.79	$(18,10^{-5})$	2	3-3	74.99±0.24	75.82±--
MUS	92.26±0.22	$(30.0,10^{-5})$	3	10.0-10.0-10.0	91.58±0.36	$(27,10^{-4})$	3	3-3-3	85.11±0.02	85.26±0.02
EGS	92.52±0.47	$(30.0,10^{-5})$	3	10.0-9.9-9.7	93.17±0.04	$(18,10^{-3})$	3	3-3-3	96.80±0.13	97.60±--
NUR	46.55±1.82	$(40.0,10^{-5})$	4	10.0-10.0-10.0-10.0	44.61±2.34	$(36,10^{-5})$	4	3-3-3-3	44.85±--	44.59±--
HTR	93.99±0.04	$(47.0,10^{-5})$	5	10.0-10.0-9.5-9.5-9.0	93.49±0.04	$(45,10^{-5})$	5	3-3-3-3-3	90.71±0.02	91.60±0.28
AVI	28.85±--	$(45.4,10^{4})$	5	15.4-13.8-10.2-4.0-2.0	28.12±2.63	$(36,10^{4})$	4	3-3-3-3	29.51±0.51	30.33±0.02
CCC	79.82±0.15	$(27.5,10^{-5})$	3	9.5-9.0-9.0	83.61±0.02	$(27,10^{-4})$	3	3-3-3	82.42±0.07	82.47±0.04
ADU	92.74±0.05	$(112.5,10^{-2})$	2	58.5-54.0	93.07±0.07	$(45,10^{-5})$	2	8-7	84.39±0.03	84.07±0.06
SHU	74.90±--	$(155.5,10^{-5})$	5	50.5-43.0-31.0-20.5-10.5	75.20±--	$(75,10^{-5})$	5	7-6-5-4-3	78.38±0.13	78.25±0.03
INF	92.28±--	$(144.0,10^{-5})$	2	75.5-68.5	93.71±--	$(51,10^{-4})$	2	9-8	94.68±0.51	95.58±--
CON	57.82±0.26	$(380,10^{-5})$	2	200.0-180.0	52.23±0.34	$(81,10^{-5})$	2	14-13	65.53±0.04	65.68±0.23
CLD	52.14±0.05	$(327,10^{-5})$	5	115.0-80.0-60.0-50.0-22.0	52.48±0.05	$(123,10^{-5})$	5	10-10-8-6-7	47.92±0.13	47.65±0.11
CLS	38.60±0.07	$(270,10^{-4})$	3	97.0-91.0-82.0	42.31±0.23	$(87,10^{-4})$	3	10-10-9	46.91±0.40	47.29±0.29
SKI	93.77±0.01	$(182,10^{-5})$	2	102.0-80.0	93.71±0.04	$(63,10^{-5})$	2	11-10	92.48±0.42	92.10±--
COV	75.88±0.08	$(190.5,10^{-5})$	3	65.0-65.0-60.5	77.33±0.16	$(72,10^{-5})$	3	8-8-8	76.31±0.59	76.32±--
KDD	63.18±0.67	$(221,10^{-5})$	4	61.0-56.0-53.0-51.0	63.08±0.51	$(87,10^{-5})$	4	8-7-7-7	48.63±0.01	48.47±0.05
POK	61.53±0.01	$(185,10^{-5})$	2	100.0-85.0	59.45±0.05	$(57,10^{-5})$	2	10-9	54.19±0.04	54.35±0.05

According to Table III, when FTA attempts to realize fast training of our recent deep fuzzy classifier DSA-FC [1], we may be a little afraid that its classification performance becomes very inferior to the original learning algorithm. Fortunately, the following statistical results by using recently-developed statistical method—Wilcoxon signed-ranks test [42-43] on the obtained average testing accuracies in Table III can help us resolve such a concern. As a non-parametric statistical method based on ranking the differences between the performances of two classifiers, the Wilcoxon signed-ranks test consists of the following steps [42] with the null-hypothesis that the two adopted algorithms perform equally well on all the adopted real-world datasets.

Step 1: As for the "Ranks" column in Table IV, we first rank the differences (denoted by d_i (see Table IV)) between the average testing accuracies (see Table III) of both DSA-FC and FTA on each adopted dataset according to their absolute values from the smallest to the largest value. It should be noted that the average ranks are assigned to the same absolute difference in the case of a tie [42-43]. For example, both DSA-FC and FTA have the same absolute difference on *WAV* and *NUR* being 1.94, the corresponding ranks are 12 and 13. The sum of the two ranks is 25. As a result, we can get a mean rank of 12.5 which is assigned to both the absolute differences of 1.94.

Step 2: Based on the "d_i" column in Table IV, we then calculate the sum of positive ranks, denoted as R^+, on the real-world datasets on which FTA outperforms DSA-FC, also the sum of negative ranks, denoted as R^-, for the opposite. If there exist $d_i = 0$, the corresponding ranks should be added to R^+ and R^-. Therefore, R^+ and R^- are respectively expressed as

$$R^+ = \sum_{d_i>0} \text{rank}(d_i) + \frac{1}{2}\sum_{d_i=0} \text{rank}(d_i) \tag{35}$$

and

$$R^- = \sum_{d_i<0} \text{rank}(d_i) + \frac{1}{2}\sum_{d_i=0} \text{rank}(d_i). \tag{36}$$

Please note, if there exists an odd number of $d_i = 0$, one of them is ignored.

Step 3: Lastly, the null-hypothesis is rejected if the minimum value of R^+ and R^- is equal or less than the baseline critical value which is determined based on the number of adopted datasets and a confidence level α. Otherwise, the null-hypothesis cannot be rejected [42-43].

From Table IV, we can obtain that $R^+ = 73.0$ and $R^- = 98.0$, therefore, the minimum of R^+ and R^- is $R^+ = 73.0$. In addition, the number of all the real-world datasets is 18, and the confidence level α is taken as $\alpha = 0.05$, the baseline critical value is 40 [42-43]. Therefore, the null-hypothesis cannot be rejected because the minimum value $R^+ = 73.0$ is bigger than 40. Hence, based on the above statistical results on Table IV, we can conclude that FTA keeps comparable testing performance to

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

DSA-FC on all the adopted datasets. With these statistical results, now according to both Table III and our experimental records, we have the following more observations.

TABLE IV
COMPARISON OF CLASSIFICATION PERFORMANCES OF BOTH DSA-FC AND FTA BASED ON WILCOXON SIGNED-RANKS TEST

Datasets	DSA-FC	FTA	Differences (d_i)	Absolute Differences	Ranks
WAV	68.73	66.79	-1.94	1.94	12.5
PHO	79.40	76.48	-3.29	2.92	15
MUS	92.26	91.58	-0.68	0.68	8
EGS	92.52	93.17	+0.65	0.65	7
NUR	46.55	44.61	-1.94	1.94	12.5
HTR	93.99	93.49	-0.50	0.50	6
AVI	28.85	28.12	-0.73	0.73	9
CCC	79.82	83.61	+3.79	3.79	17
ADU	92.74	93.07	+0.33	0.33	4
SHU	74.90	75.20	+0.30	0.30	3
INF	92.28	93.71	+1.43	1.43	10
CON	57.82	52.23	-5.59	5.59	18
CLD	52.14	52.48	+0.34	0.34	5
CLS	38.60	42.31	+3.71	3.71	16
SKI	93.77	93.71	-0.06	0.06	1
COV	75.88	77.33	+1.45	1.45	11
KDD	63.18	63.08	-0.10	0.10	2
POK	61.53	59.45	-2.08	2.08	14

(1) According to $\bar{R}$'s values in Table III, FTA generates a little less fuzzy rules than DSA-FC in the resultant deep fuzzy classifiers on the first eight small-scale datasets, i.e., *WAV*, *PHO*, *MUS*, *EGS*, *NUR*, *HTR*, *AVI* and *CCC*. However, with the increase of the total number of data samples, FTA generates less and even much less fuzzy rules than DSA-FC for the resultant sub-classifier at each layer, especially on the last ten datasets, while keeping comparable classification performance to DSA-FC. That is, FTA indeed makes the resultant deep fuzzy classifier to have better generalization capability.

(2) For both DSA-FC and FTA on all the adopted datasets, the total number of the layers in the resultant deep fuzzy classifiers generally ranges from 2 to 5, which is in line with the observations in our recent work about DSA-FC [1]. However, too many layers may bring damage to the manifold structure of the original inputs and hence the testing performances of both DSA-FC and FTA may accordingly not be improved and even may have overfitting phenomenon. Due to the space limitation of this paper, here we take the datasets *MUS* and *CON* as an example. According to our experimental records, with five layers, the average testing accuracy by DSA-FC for the sub-classifier at each layer is "91.24±0.20", "91.15±0.35", "92.26±0.22", "92.12±0.16" and "92.24±0.30", respectively, on *MUS*, and "56.42±0.11", "57.82±0.26", "53.78±0.10", "56.24±0.34" and "56.91±0.29", respectively, on *CON*. The average testing accuracy by FTA for the sub-classifier at each layer is "90.56±0.44", "90.48±0.54", "91.58±0.36", "91.32±0.41" and "91.48±0.57", respectively, on *MUS*, and "51.85±0.24", "52.23±0.34", "52.05±0.36", "49.89±0.23" and "50.99±0.55", respectively, on *CON*. Both the two adopted algorithms obtain the best average testing accuracies at the third layer on *MUS* and the second layer on *CON*. After that, embedding more smooth information into the original inputs, with the increase of the layers, cannot lift the testing performances of the two adopted algorithms. The numbers of layers of the resultant deep fuzzy classifiers by the two adopted algorithms are recommended to be set from 2 to 5 in most cases according to our expensive experiments.

TABLE V
TRAINING TIME (SECONDS) FOR EACH OF THE ADOPTED ALGORITHMS

Datasets	DSA-FC	FTA	Bernoulli dropout	Gaussian dropout
WAV	2.9387±0.0652	2.1219±0.0373	1.4323±0.0010	4.6865±0.0013
PHO	0.5675±0.0506	0.1419±0.0308	1.7759±0.0417	3.5935±0.0231
MUS	287.6500±1.9167	152.6839±1.1677	33.9761±3.2987	119.2712±0.5938
EGS	2.2375±0.0862	1.2005±0.0295	3.6713±0.0651	13.4254±0.0402
NUR	1.9117±0.0467	0.9250±0.0361	2.6581±0.0070	7.1263±0.0214
HTR	2.7109±0.0484	1.6263±0.0180	8.3276±0.0361	25.2971±0.1300
AVI	5.7070±0.1398	2.5568±0.0586	8.1916±0.0419	24.3157±0.0848
CCC	16.0078±0.1875	8.8307±0.0616	7.8355±0.0310	23.7411±0.0512
ADU	51.2547±0.5670	34.1526±0.4862	36.4238±0.1554	115.3221±0.3628
SHU	53.1094±0.4313	23.3776±0.0740	63.8168±1.7194	251.4211±3.0453
INF	44.8867±0.2402	19.5047±0.1450	58.0331±0.3578	683.9503±0.0458
CON	1.7950e+03±1.3184	696.4922±0.5755	564.1212±3.0321	2.1420e+03±14.3522
CLD	44.1250±0.1484	16.3346±0.0758	378.1052±2.1351	782.9235±2.3578
CLS	293.3281±0.4766	114.6911±0.3403	176.0999±1.2629	670.0355±2.6825
SKI	44.8047±0.3477	20.1211±0.2988	267.3580±1.0483	105.6186±0.1168
COV	9.5120e+03±54.6224	3.9947e+03±21.7765	1.1947+03±1.4372	2.7779e+03±1.7327
KDD	5.6036e+03±2.5411	2.4568e+03±1.6870	1.2817e+03±17.4792	2.8358e+03±35.0315
POK	757.0977±0.6270	305.0273±0.3496	2.0364e+03±2.0365	4.2775e+03±18.8058

(3) Except for the above statistical results, FTA actually obtains better testing accuracies than DSA-FC on *EGS*, *CCC*, *ADU*, *SHU*, *INF*, *CLD*, *CLS* and *COV*, and keeps comparable testing accuracies to DSA-FC on *MUS*, *HTR*, *AVI*, *SKI* and *KDD*. Please note, according to the column denoted as "*k*" in Table III, the value of *k* is actually set to be equal to $\sqrt{R}$, following [38-39], which is much less than the number of all generated fuzzy rules *R* used in the sub-classifier at each layer of DSA-FC. Although the top-*k* fuzzy rules are selected from all generated fuzzy rules to train the sub-classifier at each layer, FTA can still keep comparable or even better classification performance to DSA-FC. This is in accordance with our theoretical justification in Subsection III.B. In other words, the implementation of top-*k* fuzzy rules based on the gradient

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

guided learning indeed experimentally brings less co-adaption capability between fuzzy rules and hence better generalization capability of the resultant deep fuzzy classifier by FTA.

Since FTA trains the sub-classifier at each layer using gradient guided learning with top-*k* fuzzy rules, the training of each sub-classifier is simplified and hence the whole training of FTA on each dataset will become desirably fast. Table V records the training time of both DSA-FC and FTA on all the adopted datasets. In addition, Table V also reports the training time of Bernoulli and Gaussian dropouts on all the adopted datasets. According to Table V, we have the following observations:

(1) Due to the use of gradient guided learning with top-*k* fuzzy rules in the sub-classifier at each layer, with comparable or even better classification performance, FTA consumes a little less time than DSA-FC when training on *WAV*, *PHO*, *EGS*, *NUR*, *HTR* and *AVI*. What is more, this phenomenon becomes increasingly evident along with the increasing number of data samples or feature dimensions, e.g., *MUS*, *CCC*, *SHU*, *INF*, *CON*, *CLD*, *CLS*, *SKI*, *COV*, *KDD* and *POK*. Please note, FTA trains the sub-classifier at each layer three times while DSA-FC does it only once at each layer.

(2) Although our comparative study mainly focuses on DSA-FC and FTA, we should keep in mind that the training time of DSA-FC (hence FTA) obviously depends on the dimensions of the adopted datasets [1] when comparing DSA-FC and FTA with Bernoulli and Gaussian dropouts. This can be manifested from Table V. For example, owing to high dimensions of *MUS*, *CON*, *COV* and *KDD*, DSA-FC and/or FTA occupy more training time than Bernoulli and Gaussian dropouts in most cases for these four datasets. However, both DSA-FC and FTA require much less training time than Bernoulli and Gaussian dropouts for the adopted datasets with low dimensions, see *PHO*, *NUR*, *HTR*, *AVI*, *SHU*, *INF*, *CLD*, *CLS*, *SKI* and *POK* in most cases for these ten datasets.

C. Interpretability and Parameter Analysis

Since gradient guided learning works on the sub-classifier at each layer for better generalization capability, it is important for us to investigate how top-*k* fuzzy rules impact on the testing performance of FTA. Specially, on one hand, as stated in Subsection IV.A, there is a comparatively small search range for tuning the parameter *R* used for FTA on the first eight real-world datasets. On the other hand, we consider the paper's space limitation. Therefore, in this subsection we will mainly focus on both the average classification performance and the corresponding training time of FTA with different *k* values on the remaining ten large-scale datasets. In addition, when running FTA using top-*k* fuzzy rules with 4*k*, 6*k* and 8*k* where $k=\sqrt{R}$ on the first eight real-world datasets, the values of them are bigger than *R*. In this case, FTA degenerates into DSA-FC. Thus, we only investigate how the parameter *k* affects the testing performance of FTA on the remaining ten adopted datasets. In our experiments, we set the number of layers to be 5, the learning rate λ to be 0.001, and the total number of fuzzy rules *R* at each layer as in Table II. In addition, for fair comparison, we keep almost the same number *R* of fuzzy rules for training the sub-classifier at each layer when selecting different *k* values. Table VI reports the average testing accuracies and training time of FTA with *k*, 2*k*, 4*k* and 8*k*, where $k=\sqrt{R}$, on the adopted large-scale datasets. According to Table VI, average testing accuracies are slightly enhanced and the training time evidently increases in most cases, with the increase of the *k* value in the sub-classifier at each layer, which actually implies that *k* should not be a relatively big value. Please note, for example, when $8k>R$ for some datasets, the construction of each sub-classifier by FTA is the same as that by DSA-FC with *R*, for three times. In order to achieve good tradeoff between the testing performance and training time of

TABLE VI
AVERAGE TESTING ACCURACIES AND AVERAGE TRAINING TIME (SECONDS) OF FTA WITH DIFFERENT VALUES OF *k* ON THE LARGE-SCALE DATASETS

Training times		FTA (*k*)	FTA (2*k*)	FTA (4*k*)	FTA (6*k*)	FTA (8*k*)
ADU	*Acc*	93.07±0.07	93.74±0.06(+0.67)	93.76±0.05(+0.69)	94.24±0.03(+1.17)	94.60±0.03(+1.53)*
	T	34.1526±0.4862	43.1276±1.0625	52.3646±0.4466	66.0924±1.0931	77.2201±0.5501
SHU	*Acc*	75.20±--	75.23±0.00(+0.03)	74.92±0.00(-0.28)	75.24±0.00(+0.04)	75.00±0.00(-0.20)*
	T	23.3776±0.0740	30.5456±0.2346	40.3646±0.3625	49.0000±0.1432	54.5117±0.5445
INF	*Acc*	93.71±--	93.50±--(-0.21)	93.58±--(-0.13)	93.57±--(-0.14)	93.61±0.00(-0.10)
	T	19.5047±0.1450	25.8633±0.1257	30.1159±0.3262	36.9297±0.0638	44.4139±0.1289
CON	*Acc*	52.23±0.34	52.35±0.27(+0.12)	53.53±0.26(+1.30)	56.22±0.67(+3.99)	56.85±0.58(+4.62)
	T	696.4922±0.5755	752.9570±6.1081	906.9948±0.7370	1.0639e+03±1.0221	1.2322e+03±1.5182
CLD	*Acc*	52.48±0.05	51.95±0.08(-0.53)	52.09±0.10(-0.39)	52.49±0.03(+0.01)	52.66±0.01(+0.18)
	T	16.3346±0.0758	17.8490±0.0901	22.7314±0.1385	28.3255±0.1318	33.6172±0.1078
CLS	*Acc*	42.31±0.23	40.23±0.06(-2.08)	41.43±0.23(-0.88)	39.48±0.29(-2.83)	40.16±0.71(-2.15)
	T	114.6911±0.3403	133.5013±0.2704	168.8151±0.3151	206.4401±0.1667	247.2344±0.3915
SKI	*Acc*	93.71±0.04	93.78±0.02(+0.07)	93.80±0.01(+0.09)	93.79±0.01(+0.08)	93.76±0.03(+0.05)*
	T	20.1211±0.2988	27.8607±0.1999	35.0156±0.1367	39.9193±1.8372	48.0286±0.2057
COV	*Acc*	77.33±0.16	77.97±0.49(+0.64)	77.36±0.25(+0.03)	77.51±0.28(+0.18)	77.40±0.22(+0.07)
	T	3.9947e+03±21.7765	4.2410e+03±1.2266	5.5730e+03±1.6328	7.0423e+03±1.8641	8.4991e+03±3.9705
KDD	*Acc*	63.08±0.51	63.13±--(+0.05)	63.05±0.02(-0.03)	63.16±0.02(+0.08)	63.20±0.05(+0.12)*
	T	2.4568e+03±1.6870	2.8836e+03±2.7096	3.8470e+03±5.9948	4.8384e+03±2.9082	5.8358e+03±3.0378
POK	*Acc*	59.45±0.05	59.96±0.04(+0.51)	60.98±0.04(+1.53)	61.31±0.02(+1.86)	61.38±0.02(+1.93)
	T	305.0273±0.3496	326.9310±0.8665	413.1966±0.1660	513.4505±0.3437	631.2057±0.3919

* denotes the case of $8k>R$.

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

FTA, according to our extensive experiments, we recommend that an appropriate *k* value (*k*>2) may be set to be $\sqrt{R}$.

Finally let us quantitatively analyze the interpretability of the resultant deep fuzzy classifier by FTA from the perspective of the model complexity [1]. In our recent work [1], DSA-FC maintains high interpretability by mapping each feature of a data sample into five fixed fuzzy partitions, i.e., {*very low*, *low*, *medium*, *high*, *very high*}. Therefore, each fuzzy rule used in the sub-classifier at each layer becomes an interpretable fuzzy rule. What is more, due to the stacked structure [1,7-9], it requires less fuzzy rules to train DSA-FC than the zero-order or first-order fuzzy classifiers. As a result, DSA has good interpretability since DSA-FC involves less parameters to be trained in terms of the model complexity [1]. As stated in [1], the numbers of the parameters to be trained in the IF-parts and THEN-parts of each interpretable first-order fuzzy rule in the sub-classifier at each layer are respectively $2R_l d$ and $R_l(d+1)$. As a result, the total number of the parameters to be trained in DSA-FC is $\sum_{l=1}^{L}\left[R_l(3d+1)\right]$. Let us keep in mind that FTA can keep DSA-FC's stacked structure based on adversarial learning [1-6], while it uses top-*k* fuzzy rules selected from all generated fuzzy rules to train the sub-classifier three times at each layer. Therefore, the total number of the parameters to be trained in FTA is $3\sum_{l=1}^{L}\left[K_l(3d+1)\right]$ in which K_l denotes the corresponding *k* value at the *l*th layer. For the first eight datasets where the numbers of data samples or feature dimensions are obviously less than those of the remaining ten datasets, the number of the parameters trained by FTA is a little less than that trained by DSA-FC. Take the dataset *NUR* as an example. According to Table I, the number of feature dimensions in *NUR* is 8. According to Table III, DSA-FC needs four layers to obtain its best average testing accuracy of 46.55%. The number of fuzzy rules generated to train the sub-classifier at each layer is 10 according to the "R_l" column. Therefore, the parameters to be trained by DSA-FC is $(10+10+10+10)*(3*8+1)=1000$. As for FTA, it also needs four layers to obtain its best average testing accuracy of 44.61%. The number of fuzzy rules used to train the sub-classifier at each layer is 3 according to the "*k*" column. Therefore, the parameters to be trained by FTA is $3*(3+3+3+3)*(3*8+1)=900$. However, for the remaining ten large-scale datasets, the numbers of parameters trained by FTA are evidently less than those trained by DSA-FC. Take the dataset *COV* as an example, as reported in Table III, DSA-FC needs three layers to obtain the best average testing accuracy of 75.88%. The numbers of the generated fuzzy rules at these three layers are 65, 65 and 60, respectively. Hence, the total number of parameters involved in DSA-FC is $(65+65+60)*(3*54+1)=30970$. FTA needs three layers to obtain the best average testing accuracy of 77.33%. However, due to the use of top-*k* fuzzy rules, the total numbers of fuzzy rules at these three layers are 8, 8 and 8, respectively, as listed in the "*k*" column in Table III. As a result, the total number of the parameters involved in FTA is $3*(8+8+8)*(3*54+1)=11736$. In other words, FTA uses much less parameters than DSA-FC. Please note, FTA trains the sub-classifier at each layer for three times. In terms of both the number of fuzzy rules and the number of the parameters (i.e., the model complexity) in Table VII, the resultant deep fuzzy classifier by FTA has obviously better interpretability than that by DSA-FC on these large-scale datasets.

TABLE VII
MODEL COMPLEXITIES FOR BOTH DSA-FC AND FTA

Datasets	No. of Parameters by DSA-FC	No. of Parameters by FTA
WAV	2304	1152
PHO	720	288
MUS	14970	13473
EGS	1120	1080
NUR	1000	900
HTR	1175	1125
AVI	1364	1116
CCC	1890	1890
ADU	4816	1935
SHU	4312	2100
INF	4862	1734
CON	48260	10287
CLD	3270	1230
CLS	10800	3480
SKI	1820	630
COV	30970	11736
KDD	27404	10788
POK	5735	1767

V. CONCLUSIONS

Based on the proposed gradient guided learning, a fast training algorithm FTA is developed for fast training of our recently-developed deep fuzzy classifier DSA-FC [1]. Owing to both enhanced generalization capability and the maintenance of important fuzzy rules by gradient guided learning, FTA has faster training speed and better interpretability than the learning algorithm of DSA-FC in [1], while keeping comparable or even better testing accuracies especially on large-scale datasets. Our experimental results indeed indicate the above merits of FTA.

Although the generalization capability of the proposed fast training algorithm FTA has its theoretical support based on the observational quantity—the absolute values of the consequent parts $f^r(\mathbf{x})$ ($=\mathbf{p}^r\tilde{\mathbf{x}}^T$) of fuzzy rules, the other observational quantities may be studied for the selection of top-*k* fuzzy rules in near future. What is more, we will study: (1) how to appropriately set the parameter *k* in top-*k* fuzzy rules and give a reasonable guideline about it for various large-scale datasets; (2) how to extend current work into deep high-order fuzzy classifiers and/or deep neural networks, which are the researches we are doing.

REFERENCES

[1] S. Gu, F.-L. Chung, S. Wang. "A novel deep fuzzy classifier by stacking adversarial interpretable TSK sub-classifiers with smooth gradient information," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 2, pp. 361-375, Feb. 2020.

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TFUZZ.2021.3072498, IEEE Transactions on Fuzzy Systems

[2] A. Mosca, G. D. Magoulas, "Hardening against adversarial examples with the smooth gradient method," *Soft Computing*, vol. 22, no. 10, pp. 3203-3213, Jan. 2018.

[3] K. Grosse, N. Papernot, P. Manoharan *et al.*, "Adversarial perturbations against deep neural networks for malware classification," arXiv: 1606.04435v2[cs.CR], Jun. 2016.

[4] S. Gu and L. Rigazio, "Towards deep neural network architectures robust to adversarial examples," In Workshop on ICLR, 2015.

[5] T. Miyato, S.-I. Maeda, M. Koyama and S. Ishii, "Virtual adversarial training: A regularization method for supervised and semi-supervised learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 8, pp. 1979-1993, Aug. 2019.

[6] I. J. Goodfellow, J. Shlens, C. Szegedy, "Explaining and harnessing adversarial examples," In *ICLR*, 2015.

[7] D. H. Wolpert, "Stacked generalization," *Neural Netw.*, vol. 5, no. 2, pp. 241-259, Jul. 1992.

[8] T. Zhou, H. Ishibuchi and S. Wang, "Stacked blockwise combination of interpretable TSK fuzzy classifiers by negative correlation learning," *IEEE Trans. Fuzzy Syst.*, vol. 26, no. 6, pp. 3327-3341, Dec. 2018.

[9] Y. Zhang, H. Ishibuchi and S. Wang, "Deep Takagi-Sugeno-Kang fuzzy classifier with shared linguistic fuzzy rules," *IEEE Trans. Fuzzy Syst.*, vol. 26, no. 3, pp. 1535-1549, Jun. 2018.

[10] H. Yu, J. Lu and G. Zhang, "Topology learning-based fuzzy random neural network for streaming data regression," *IEEE Trans. Fuzzy Syst.*, to be published, DOI: 10.1109/TFUZZ.2020.3039681.

[11] M. Ferdaus, M. Pratama, S. Anavatti *et al.*, "PALM: An incremental construction of hyperplanes for data stream regression," *IEEE Trans. Fuzzy Syst.*, vol. 27, no. 11, pp. 2115-2129, Nov. 2019.

[12] Zuo H, Zhang G, Pedrycz W, V. Behbood; J. Lu, "Granular fuzzy regression domain adaptation in Takagi–Sugeno fuzzy models", *IEEE Trans. on Fuzzy Syst.*, vol.26, no.2, pp.847-858, 2017.

[13] Y. Deng, Z. Ren, Y. Kong *et al.*, "A hierarchical fused fuzzy deep neural network for data classification," *IEEE Trans. Fuzzy Syst.*, vol. 25, no. 4, pp. 1006-1012, Aug. 2017.

[14] D. Alistarh, T. Hoefler, M. Johansson *et al.*, "The convergence of sparsified gradient methods," in *Proc. NIPS*, 2018, pp. 5973-5983.

[15] S. Shi, Q. Wang, K. Zhao *et al.*, "A distributed synchronous SGD algorithm with global top-*k* sparsification for low bandwidth networks," in *Proc. ICDCS*, 2019, pp. 2238-2247.

[16] C. J. Mantas and J. M. Puche, "Artificial neural networks are zero-order TSK fuzzy systems," *IEEE Trans. Fuzzy Syst.*, vol. 16, no. 3, pp. 630-643, Jun. 2008.

[17] G. E. Hinton, N. Srivastava, A. Krizhevsky *et al*. (Jul. 2012). "Improving neural networks by preventing co-adaption of feature detectors." [Online]. Available: https://arxiv.org/abs/1207.0580.

[18] B. Qin, Y. Nojima, H. Ishibuchi, S. Wang, "Realizing deep high-order TSK fuzzy classifier by ensembling interpretable zero-order TSK fuzzy classifier," *IEEE Trans. Fuzzy Syst.*, to be published, DOI: 10.1109/TFUZZ.2020.3022574.

[19] N. Srivastava, G. Hinton, A. Krizhevsky *et al.*, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929-1958, Jun. 2014.

[20] A. Poernomo and D.-K. Kang, "Biased dropout and crossmap dropout: Learning towards effective dropout regularization in convolutional neural network," *Neural Netw.*, vol. 104, pp. 60-67, Aug. 2018.

[21] X. Shen, X. Tian, D. Tao *et al.*, "Continuous dropout," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 9, pp. 3926-3937, Sep. 2018.

[22] S. Wang and C. Manning, "Fast dropout training," in *Proc. ICML*, 2013, pp. 118-126.

[23] J. Ba and B. Frey, "Adaptive dropout for training deep neural networks," in *Proc. NIPS*, 2013, pp. 3084-3092.

[24] H. Wu and X. Gu, "Towards dropout training for convolutional neural networks," *Neural Netw.*, vol. 71, pp. 1-10, Nov. 2015.

[25] T. Zhou, F.-L. Chung, S. Wang, "Deep TSK fuzzy classifier with stacked generalization and triplely concise interpretability guarantee for large data," *IEEE Trans. Fuzzy Syst.*, vol. 25, no. 5, pp. 1207-1221, Oct. 2017.

[26] Z. Deng, Y. Jiang, F. L. Chung, H. Ishibuchi, and S. Wang, "Knowledge-leverage-based fuzzy system and its modeling," *IEEE Trans. Fuzzy Syst.*, vol. 21, no. 4, pp. 597-609, Aug. 2013.

[27] Z. Deng, K.-S. Choi, F.-L. Chung, S. Wang, "Scalable TSK fuzzy modeling for very large datasets using minimal-enclosing-ball approximation," *IEEE Trans. Fuzzy Syst.*, vol. 19, no. 2, pp. 210-226, Apr. 2011.

[28] T. Takagi and M. Sugeno, "Fuzzy identification of systems and its application to modeling and control," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-15, no. 1, pp. 116-132, Jan. 1985.

[29] K. Nozaki, H. Ishibuchi and H. Tanaka, "A simple but powerful heuristic method for generating fuzzy rules from numerical data," *Fuzzy Sets and Systems*, vol. 86, no. 3, pp. 251-270, Mar. 1997.

[30] K. Kilic, O. Uncu, and I. B. Turksen, "Comparison of different strategies of utilizing fuzzy clustering in structure identification," *Inf. Sci.*, vol. 177, pp. 5153-5162, Dec. 2007.

[31] M. Elkano, J. A. Sanz, E. Barrenechea *et al.*, "CFM-BD: A distributed rule induction algorithm for building compact fuzzy models in big data classification problems," *IEEE Trans. on Fuzzy Syst.*, vol. 28, no. 1, pp. 163-177, Jan. 2020.

[32] D. Bacciu and F. Crecchi, "Augmenting recurrent neural networks resilience by dropout," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 1, pp. 345-351, Jan. 2020.

[33] Y. Jiang, J. Fan, T. Chai *et al.*, "Tracking control for linear discrete-time networked control systems with unknow dynamics and dropout," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 10, pp. 4607-4620, Oct. 2018.

[34] B. Qin, F.-L. Chung and S. Wang, "Biologically plausible fuzzy-knowledge-out and its induced wide learning of interpretable TSK fuzzy classifiers," *IEEE Trans. Fuzzy Syst.*, vol. 28, no. 7, pp. 1276-1290, Jul. 2020.

[35] Y. Jiang, Z. Deng, Shitong Wang *et al.*, "Recognition of epileptic EEG signals using a novel multiview TSK fuzzy system," *IEEE Trans. Fuzzy Syst.*, vol. 25, no. 1, pp. 3-20, Feb. 2017.

[36] A. Frank, A. Asuncion. (2010). *UCI Machine Learning Repository*. [Online]. Available: http://archive.ics.uci.edu/ml.

[37] J. Alcalá-Fdez, A. Fernández, J. Luengo, J. Derrac, S. García, "KEEL data-mining software tool: Data set repository, integration of algorithms and experimental analysis framework," *Journal of Multiple-Valued Logic and Soft Computing*, vol. 17, nos. 2-3, pp. 255-287, Jan. 2011.

[38] Y. Wang, S. Xia, X. Zhu *et al.*, "A novel consistent random forest framework: Bernoulli random forests," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3510-3523, Aug. 2018.

[39] L. Breiman, "Manual on setting up, using, and understanding random forests V3.1," Statistics Dept. Univ. California, Berkeley, CA, USA, Tech. Rep., 2002.

[40] I. F. Alves and C. Neves, "Extreme value distributions," In International Encyclopedia of Statistical Science, pp. 493-496, Springer Berlin Heidelberg.

[41] W. E. Weisstein, "Extreme Value Distribution," [online]. http://mathworld.wolfram.com/ExtremeValueDistribution.html.

[42] J. Demsar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1-30, Jan. 2006.

[43] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics*, vol. 1, no. 6, pp. 80-83, Dec. 1945.

[44] G. Wang, K. Choi, J. Lu *et al.*, "Deep additive least squares support vector machines for classification with model transfer," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 49, no. 7, pp. 1527-1540, Jul. 2019.

[45] G. Wang, J. Lu, K. Choi and G. Zhang, "A Transfer-Based Additive LS-SVM Classifier for Handling Missing Data," *IEEE Trans. Cybern.*, vol. 50, no. 2, pp. 739-752, Feb. 2020.

1063-6706 (c) 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Prince Edward Island. Downloaded on May 15,2021 at 06:23:36 UTC from IEEE Xplore. Restrictions apply.